

Notes PYTHON

Chapter 1: `print()` Function in Python

Definition

The `print()` function in Python is a built-in function used to display output on the screen.

Basic Examples

Print a String

```
print("I am learning Python")
```

Output

```
I am learning Python
```

Print Any Data Type

```
print(4 + 5)
```

Output

```
9
```

Print Multiple Objects

```
print("I love coding", 1 + 2, True)
```

Output

```
I love coding 3 True
```

Print Multiple Lines

```
print("I am a data scientist")  
print(5 + 6)  
print(False)
```

Output

```
I am a data scientist  
11  
False
```

General Syntax

```
print(*objects, sep=' ', end='\n', file=sys.stdout, flush=False)
```

Parameter	Meaning
<code>*objects</code>	Values to print
<code>sep</code>	Separator between values
<code>end</code>	What to print at the end
<code>file</code>	Output location
<code>flush</code>	Forces output immediately

Important Parameters

1. `sep` Parameter

Used to separate multiple values.

```
print("2025", "06", "03", sep="-")
```

Output

```
2025-06-03
```

2. `end` Parameter

Used to change the ending character.

```
print("Loading...", end=" ")  
print("100%")
```

Output

```
Loading... 100%
```

3. Printing to a File

```
print("Print this to a file", file=open("output.txt", "w"))
```

This writes the output into `output.txt`.

`flush=True`

Without **`flush=True`**

```
import time

print("Loading...", end=" ")
time.sleep(2)
print("Done!")
```

Output

```
Loading... Done!
```

Sometimes `"Loading..."` may not appear immediately because Python stores output temporarily in a buffer.

With `flush=True`

```
import time

print("Loading...", end=" ", flush=True)
time.sleep(2)
print("Done!")
```

Output

```
Loading... Done!
```

Now `"Loading..."` appears instantly.

What is Flushing?

Flushing means forcing Python to display output immediately.

Normally, Python stores output in a temporary memory area called a buffer and prints it later.

The buffer is cleared when:

- The buffer becomes full
 - A newline (`\n`) is printed
 - The program ends
 - `flush=True` is used
-

Key Point

```
flush=True
```

This tells Python:

Display the output immediately without waiting.

Chapter 2: Comments in Python

Definition

Comments are notes written inside the code to explain what the code does.

Python ignores comments during execution.

Comments help programmers understand the code easily.

Types of Comments

1. Single-Line Comment

A single-line comment starts with the `#` symbol.

```
# This is a single-line comment
```

```
x = 5 # Assign 5 to x
```

2. Multi-Line Comment

Python does not have a special syntax for multi-line comments.

We use multiple `#` symbols for writing comments on several lines.

```
# This is a multi-line comment  
# written over  
# several lines
```

Important Points

- Comments are ignored by Python
- Comments improve code readability
- `#` is used for comments
- Comments are useful for explaining logic and code

Chapter 3: Data Types in Python

Definition

A data type defines the type of value stored in a variable.

Python is dynamically typed, so we do not need to declare the data type manually.

Types of Data in Python

Python mainly has 3 types of data:

Type	Examples
Basic Data Types	<code>int</code> , <code>float</code> , <code>complex</code> , <code>bool</code> , <code>str</code>
Container Data Types	<code>list</code> , <code>tuple</code> , <code>set</code> , <code>dict</code>
User-Defined Data Types	<code>class</code>

Basic Data Types

1. Integer (`int`)

Stores whole numbers.

Examples

```
5  
-6  
0
```

2. Float (`float`)

Stores decimal numbers.

Examples

```
1.5  
-2.6  
5.699
```

3. Complex (`complex`)

Stores complex numbers.

Examples

```
1 + 5j  
2.5 + 6.2j  
8j
```

4. Boolean (`bool`)

Stores only two values.

Values

```
True  
False
```

5. String (`str`)

Stores text data.

Examples

```
"Python"  
'I love coding'  
"122"
```

Container Data Types

1. List (`list`)

Ordered and changeable collection.

Example


```
[1, "coding", True]
```

2. Tuple (**tuple**)

Ordered but unchangeable collection.

Example

```
(1, "coding", True)
```

3. Set (**set**)

Unordered collection of unique values.

Example

```
{1, "coding", True}
```

4. Dictionary (**dict**)

Stores data in key-value pairs.

Example

```
{  
    "Name": "Debottam",  
    "Age": 24,  
    "Married": False  
}
```

User-Defined Data Type

Class (**class**)

Used to create custom data types.

Example

```
class Student:  
    pass
```

type() Function

The `type()` function is used to check the data type of a value or variable.

Example 1

```
type(1)
```

Output

```
int
```

Example 2

```
type([1, 2, "code"])
```

Output

```
list
```

Example 3

```
type(5.6 + 9j)
```

Output

```
complex
```

Chapter 4: Variables and Literals in Python

Variables

Definition

Variables are used to store data in Python.

We use variables when:

- The value may change in the future
- We want user input
- We want to store data temporarily

Features of Python Variables

1. Dynamically Typed

Python automatically decides the data type of a variable.

Example

```
x = 5
```

```
y = "Red"
```

Here:

- `x` is an integer (`int`)
- `y` is a string (`str`)

We do not need to declare the type manually.

2. Dynamically Bound

A variable can store different types of values at different times.

Example

```
x = 10  
x = "hello"
```

First, `x` stores an integer.

Later, `x` stores a string.

Python allows changing the type of value stored in a variable.

Special Syntax for Variables

Method 1: Multiple Variables in One Line

```
a, b, c = 5, "Red", 1.67  
  
print(a)  
print(b)  
print(c)
```

Output

```
5
Red
1.67
```

Method 2: Using Semicolon

```
a = 5; b = "Red"; c = 1.67

print(a)
print(b)
print(c)
```

Output

```
5
Red
1.67
```

Method 3: Same Value to Multiple Variables

```
a = b = 4
c = "Red"

print(a)
print(b)
print(c)
```

Output

```
4
4
```

Red

Literals

Definition

A literal is a fixed value written directly in the code.

Literals represent constant data.

Examples

```
x = 10
name = "Bob"
```

Here:

- `10` is a numeric literal
- `"Bob"` is a string literal

Types of Literals

1. Numeric Literals

Used for numbers.

Type	Examples
Integer	<code>100</code> , <code>-7</code>
Float	<code>3.14</code> , <code>-0.5</code>
Complex	<code>2 + 3j</code>

2. String Literals

Text written inside quotes.

Examples

```
"Hello"  
'Python'  
'''Multi-line string'''
```

3. Boolean Literals

Used in conditions and logic.

Values

```
True  
False
```

4. Special Literal: `None`

`None` means no value.

It is used as a placeholder or empty value.

Example

```
x = None
```

Chapter 5: Keywords and Identifiers in Python

Keywords

Definition

Keywords are reserved words in Python.

They have special meanings and cannot be used as variable names, function names, or identifiers.

Python has 35 keywords.

List of Keywords

```
import keyword

print(keyword.kwlist)
```

Output

```
['False', 'None', 'True', 'and', 'as', 'assert', 'async',  
'await', 'break', 'class', 'continue', 'def', 'del',  
'elif', 'else', 'except', 'finally', 'for', 'from',  
'global', 'if', 'import', 'in', 'is', 'lambda',  
'nonlocal', 'not', 'or', 'pass', 'raise', 'return',  
'try', 'while', 'with', 'yield']
```

Identifiers

Definition

Identifiers are names given to:

- Variables
- Functions

- Classes
 - Modules
-

Examples of Identifiers

```
name = "Alice"
```

Here, `name` is an identifier.

```
def greet():  
    pass
```

Here, `greet` is an identifier.

```
class Student:  
    pass
```

Here, `Student` is an identifier.

Rules for Valid Identifiers

- Must start with a letter or underscore `_`
- Can contain letters, digits, and underscores
- Cannot start with a number
- Cannot use Python keywords
- Identifiers are case-sensitive

Example:

```
Name ≠ name
```

Valid and Invalid Identifiers

Valid Identifiers	Invalid Identifiers
<code>name</code>	<code>lname</code>
<code>user_name</code>	<code>user-name</code>
<code>_temp</code>	<code>@temp</code>
<code>MAX_VALUE</code>	<code>my var</code>
<code>x</code>	<code>for</code>
<code>age25</code>	<code>25age</code>
<code>data_123</code>	<code>def</code>
<code>name_</code>	<code>print()</code>
<code>value2temp</code>	<code>import*</code>

Important Points

- Keywords are reserved words
- Identifiers are user-defined names
- Keywords cannot be used as identifiers
- Identifiers must follow naming rules

Chapter 6: Type Conversion in Python

Definition

Type conversion means changing one data type into another data type.

Python allows type conversion only when the conversion is valid.

Converting `int` to `str`

Example

```
x = 5

print(type(x))

x = str(x)

print(type(x))
```

Output

```
<class 'int'>
<class 'str'>
```

Invalid Type Conversion

We cannot convert every string into an integer.

Example

```
x = "Five"

print(type(x))

x = int(x)

print(type(x))
```

Output

```
<class 'str'>  
ValueError
```

This happens because `"Five"` is not a numeric value.

More Examples of Type Conversion

Convert String to List

```
x = "Hello"  
  
print(list(x))
```

Output

```
['H', 'e', 'l', 'l', 'o']
```

Convert Float to Integer

```
x = 4.7  
  
print(int(x))
```

Output

```
4
```

The decimal part is removed.

Temporary Type Conversion

Type conversion is temporary unless stored back into the variable.

Example 1

```
x = 5

str(x)

print(type(x))
```

Output

```
<class 'int'>
```

The data type remains `int`.

Example 2

```
x = 5

x = str(x)

print(type(x))
```

Output

```
<class 'str'>
```

Now the data type changes permanently because it is stored again.

Boolean Type Conversion

Python treats:

- `True` as `1`
- `False` as `0`

Example

```
x = True + 2  
  
print(x)  
print(type(x))
```

Output

```
3  
<class 'int'>
```

Taking Input From User

Example

```
name = input("Enter your name: ")
```

Important Point

By default, Python stores all user input as a string (`str`).

Problem Example

```
num1 = input("Enter 1st number: ")  
num2 = input("Enter 2nd number: ")
```

```
print(num1 + num2)
```

Input

5
6

Output

56

The numbers are treated as strings.

Solution: Convert Input to `int`

Example

```
num1 = int(input("Enter 1st number: "))  
num2 = int(input("Enter 2nd number: "))  
  
print(num1 + num2)
```

Input

5
6

Output

11

Chapter 7: Operators in Python

Definition

Operators are symbols used to perform operations on variables and values.

Python has 7 main types of operators.

1. Arithmetic Operators

Used for mathematical calculations.

Operator	Meaning	Example	Result
<code>+</code>	Addition	<code>5 + 2</code>	<code>7</code>
<code>-</code>	Subtraction	<code>5 - 2</code>	<code>3</code>
<code>*</code>	Multiplication	<code>5 * 2</code>	<code>10</code>
<code>/</code>	Division	<code>5 / 2</code>	<code>2.5</code>
<code>//</code>	Floor Division	<code>5 // 2</code>	<code>2</code>
<code>%</code>	Modulus	<code>5 % 2</code>	<code>1</code>
<code>**</code>	Exponent	<code>2 ** 3</code>	<code>8</code>

2. Comparison Operators

Used to compare values.

Operator	Meaning	Example	Result
<code>==</code>	Equal to	<code>5 == 5</code>	<code>True</code>
<code>!=</code>	Not equal to	<code>5 != 2</code>	<code>True</code>
<code>></code>	Greater than	<code>5 > 3</code>	<code>True</code>

Operator	Meaning	Example	Result
<	Less than	2 < 3	True
>=	Greater than or equal	5 >= 5	True
<=	Less than or equal	4 <= 2	False

3. Assignment Operators

Used to assign values to variables.

Operator	Meaning	Example
=	Assign value	x = 5
+=	Add and assign	x += 3
-=	Subtract and assign	x -= 2
*=	Multiply and assign	x *= 4
/=	Divide and assign	x /= 2
//=	Floor divide and assign	x //= 2
%=	Modulus and assign	x %= 2
**=	Power and assign	x **= 2

4. Logical Operators

Used with boolean values (True and False).

Operator	Meaning	Example	Result
and	True if both are true	True and False	False
or	True if one is true	True or False	True
not	Reverse the result	not True	False

5. Bitwise Operators

Used at binary level.

Operator	Meaning	Example	Result
<code>&</code>	AND	<code>5 & 3</code>	<code>1</code>
<code> </code>	OR	<code>5 3</code>	<code>7</code>
<code>^</code>	XOR	<code>5 ^ 3</code>	<code>6</code>
<code>~</code>	NOT	<code>~5</code>	<code>-6</code>
<code><<</code>	Left Shift	<code>5 << 1</code>	<code>10</code>
<code>>></code>	Right Shift	<code>5 >> 1</code>	<code>2</code>

6. Membership Operators

Used to check whether a value exists in a sequence.

Operator	Meaning	Example	Result
<code>in</code>	Value present	<code>'a' in 'cat'</code>	<code>True</code>
<code>not in</code>	Value not present	<code>5 not in [1,2,3]</code>	<code>True</code>

7. Identity Operators

Used to compare memory location of objects.

Operator	Meaning	Example
<code>is</code>	Same object	<code>x is y</code>
<code>is not</code>	Different object	<code>x is not y</code>

Example of Identity Operator

```
a = "Hello World"
b = "Hello World"

print(a is b)
```

Output

False

`is` checks whether both variables point to the same memory location, not just the same value.

Chapter 8: If-Else Statement in Python

Definition

The `if-else` statement is used to make decisions in Python.

Python uses indentation (spaces or tabs) to define code blocks instead of `{ }` brackets.

Basic Syntax

```
if condition:
    # Code runs if condition is True

elif another_condition:
    # Code runs if elif condition is True

else:
    # Code runs if all conditions are False
```

Example

```
x = 10

if x > 0:
    print("Positive number")

elif x == 0:
    print("Zero")

else:
    print("Negative number")
```

Output

```
Positive number
```

Important Points

- Use `elif` when multiple conditions are related
- Indentation is compulsory in Python
- Python does not use `{ }` brackets

Login System Example

Problem

- Correct Email: `debottamghosh361@gmail.com`
- Correct Password: `1234`

Conditions:

- If email and password are correct → print `"WELCOME!"`
- If email is correct but password is wrong:

- Ask for password again
 - If correct → print "IT'S CORRECT PASSWORD"
 - Otherwise → print "WRONG PASSWORD"
 - For all other cases → print "INCORRECT CREDENTIALS"
-

Program

```
cor_email = "debottamghosh361@gmail.com"
cor_pass = "1234"

email = input("Enter your email ID: ")
password = input("Enter your password: ")

if email == cor_email and password == cor_pass:
    print("WELCOME!")

elif email == cor_email and password != cor_pass:
    print("INCORRECT PASSWORD")

    password = input("Enter your password again: ")

    if password == cor_pass:
        print("IT'S CORRECT PASSWORD")
    else:
        print("WRONG PASSWORD")

else:
    print("INCORRECT CREDENTIALS")
```

One-Line If-Else (Ternary Operator)

Used to write simple `if-else` statements in one line.

Example

```
num = int(input("Enter a number: "))  
  
result = "Even" if num % 2 == 0 else "Odd"  
  
print(result)
```

How It Works

- If the condition is `True`, `"Even"` is stored
- Otherwise, `"Odd"` is stored

Chapter 9: Loops in Python

Definition

Loops are used to repeat a block of code multiple times.

Types of Loops

Python has two types of loops:

1. `while` loop
 2. `for` loop
-

1. While Loop

The `while` loop runs as long as the condition is `True`.

Syntax

```
while condition:  
    # code block
```

Example

```
x = 3  
  
while x > 0:  
    print(x)  
    x -= 1
```

Output

```
3  
2  
1
```

Table Program Using While Loop

```
n = int(input("Enter a number: "))  
  
print(f"LET'S LEARN THE TABLE OF {n} TODAY!")  
  
i = 1
```

```
while i <= 10:  
    print(n, "*", i, "=", n * i)  
    i += 1
```

2. For Loop

The `for` loop is used to iterate over sequences.

Sequences Used in `for` Loop

- Range
- String
- List
- Tuple
- Set
- Dictionary

Range Function

Examples

```
range(6)
```

Output:

```
0, 1, 2, 3, 4, 5
```

```
range(3, 8)
```


Output:

```
3, 4, 5, 6, 7
```

```
range(5, 16, 3)
```

Output:

```
5, 8, 11, 14
```

Syntax of For Loop

```
for item in sequence:  
    # code block
```

Examples of For Loop

1. Using Range

```
for i in range(7):  
    print(i, end=" ")
```

Output

```
0 1 2 3 4 5 6
```

2. Using String

```
for i in "Python":  
    print(i, end=" ")
```

Output

P y t h o n

3. Using Tuple

```
for i in (3, 5, 7, 10):  
    print(i, end=" ")
```

Output

3 5 7 10

4. Using List

```
for i in [1, 3, 9]:  
    print(i, end=" ")
```

Output

1 3 9

Loop Control Statements

Statement	Meaning
<code>break</code>	Stops the loop

Statement	Meaning
<code>continue</code>	Skips current iteration
<code>pass</code>	Does nothing

1. `break`

Used to stop the loop immediately.

Example

```
for i in range(5):  
  
    if i == 3:  
        break  
  
    print(i, end=" ")
```

Output

```
0 1 2
```

2. `continue`

Skips the current iteration and moves to the next iteration.

Example

```
for i in range(5):  
  
    if i == 3:  
        continue
```

```
print(i, end=" ")
```

Output

```
0 1 2 4
```

3. **pass**

Does nothing. Used as a placeholder.

Example

```
for i in range(5):  
  
    if i == 3:  
        pass  
  
    print(i, end=" ")
```

Output

```
0 1 2 3 4
```

Example Using **break**, **continue**, and **pass**

```
for i in range(10, 20):  
  
    print("\n", i, end=" ")
```

```
if i == 13:
    continue

elif i == 16:
    pass
    print("- pass", end=" ")

elif i == 19:
    break
```

Explanation

- `continue` skips the remaining code of the current iteration
- `pass` does nothing
- `break` stops the loop completely

Nested Loops

Nested loops are loops inside another loop.

They are commonly used for pattern printing.

Star Pattern Example

```
*
**
***
****
*****
```

Program

```
n = 5

for i in range(1, n + 1):

    for j in range(i):
        print("*", end=" ")

    print()
```

When to Use **for** and **while**

Use **for** Loop

- When the number of iterations is known

Use **while** Loop

- When the number of iterations is unknown
- When the loop depends on a condition

Chapter 10: Guess The Number Game in Python

Definition

Guess The Number is a simple game where the computer selects a random number and the user tries to guess it.

The game gives hints after every guess.

Rules of the Game

- The computer chooses a random number between 1 and 100
- The user guesses the number
- The program checks the guess and gives hints

Possible Hints

- "Too low" → Guess a higher number
- "Too high" → Guess a lower number
- "Correct" → You guessed the right number

The game continues until the correct number is guessed.

The program also counts the number of attempts.

Program

```
import random

cor_num = random.randint(1, 100)

num_attempt = 1

guess = int(input(f"{num_attempt} | Guess a number (1-100):
"))

while guess != cor_num:

    if guess < cor_num:
        num_attempt += 1

        guess = int(input(f"{num_attempt} | Too low! Guess hi
gher: "))
```

```
elif guess > cor_num:
    num_attempt += 1

    guess = int(input(f"{num_attempt} | Too high! Guess lower: "))

print()
print(f"You guessed correctly in {num_attempt} attempts!")
```

Explanation

import random

Imports the random module.

random.randint(1, 100)

Generates a random number between **1** and **100**.

while guess != cor_num

The loop keeps running until the user guesses the correct number.

If Conditions

- If guess is smaller → print **"Too low"**
- If guess is larger → print **"Too high"**

Attempt Counter

num_attempt += 1

Increases the attempt count after every wrong guess.

Important Concepts Used

- Variables
- Input
- Type Conversion
- `while` Loop
- `if-elif` Conditions
- Random Module
- Comparison Operators

Chapter 11: Built-in Functions in Python

Definition

Built-in functions are functions already available in Python.

They help perform common tasks like type conversion, calculations, input/output, and working with data structures.

1. Type Conversion Functions

Used to convert one data type into another.

Function	Purpose	Example	Output
<code>int()</code>	Convert to integer	<code>int("5")</code>	<code>5</code>
<code>float()</code>	Convert to float	<code>float("3.14")</code>	<code>3.14</code>
<code>str()</code>	Convert to string	<code>str(10)</code>	<code>"10"</code>
<code>bool()</code>	Convert to boolean	<code>bool(0)</code>	<code>False</code>

2. Functions Used in Sequences

Function	Purpose	Example	Output
<code>range()</code>	Generate numbers	<code>range(5)</code>	<code>0 to 4</code>
<code>enumerate()</code>	Get index and value	<code>enumerate(['a','b'])</code>	<code>(0, 'a')</code>
<code>zip()</code>	Combine sequences	<code>zip([1,2],[3,4])</code>	<code>(1,3)</code>

3. Math and Logic Functions

Function	Purpose	Example	Output
<code>abs()</code>	Absolute value	<code>abs(-7)</code>	<code>7</code>
<code>round()</code>	Round number	<code>round(3.567, 2)</code>	<code>3.57</code>
<code>pow()</code>	Power	<code>pow(2, 3)</code>	<code>8</code>
<code>min()</code>	Smallest value	<code>min(1, 5, -2)</code>	<code>-2</code>
<code>max()</code>	Largest value	<code>max(1, 5, -2)</code>	<code>5</code>
<code>sum()</code>	Sum of values	<code>sum([1,2,3])</code>	<code>6</code>

4. Data Structure Utility Functions

Function	Purpose	Example	Output
<code>len()</code>	Find length	<code>len("hello")</code>	<code>5</code>
<code>sorted()</code>	Sort values	<code>sorted([3,1,2])</code>	<code>[1,2,3]</code>
<code>reversed()</code>	Reverse values	<code>list(reversed("abc"))</code>	<code>['c','b','a']</code>
<code>list()</code>	Convert to list	<code>list("abc")</code>	<code>['a','b','c']</code>
<code>set()</code>	Convert to set	<code>set([1,1,2])</code>	<code>{1,2}</code>
<code>dict()</code>	Create dictionary	<code>dict(a=1,b=2)</code>	<code>{'a':1,'b':2}</code>

5. Evaluation and Checking Functions

Function	Purpose	Example	Output
<code>type()</code>	Check data type	<code>type(3.14)</code>	<code><class 'float'></code>
<code>id()</code>	Get memory address	<code>id(x)</code>	Integer value
<code>isinstance()</code>	Check object type	<code>isinstance(3, int)</code>	<code>True</code>
<code>eval()</code>	Evaluate expression	<code>eval("2+3")</code>	<code>5</code>
<code>input()</code>	Take user input	<code>input("Name: ")</code>	User input

6. Other Useful Functions

Function	Purpose
<code>help()</code>	Show help/documentation
<code>dir()</code>	Show methods and attributes
<code>print()</code>	Display output
<code>open()</code>	Open a file

Module vs Package vs Library

Aspect	Module	Package	Library
Definition	Single Python file	Folder containing modules	Collection of packages/modules
Structure	<code>.py</code> file	Directory	Large toolkit
Purpose	Organize code	Organize modules	Solve bigger problems
Example	<code>math.py</code>	<code>numpy</code>	<code>TensorFlow</code> , <code>Pandas</code>

Summary

- Module → Single Python file
- Package → Collection of modules
- Library → Collection of packages and modules

Examples of Built-in Functions

enumerate()

Used to get index and value together.

Example

```
l = ['a', 1, True, 'Python']  
  
print(list(enumerate(l)))
```

Output

```
[(0, 'a'), (1, 1), (2, True), (3, 'Python')]
```

zip()

Used to combine multiple sequences.

Example

```
l1 = ['a', 1, True]  
l2 = [False, 'coding', 5]  
  
print(list(zip(l1, l2)))
```

Output

```
[('a', False), (1, 'coding'), (True, 5)]
```

round()

Used to round numbers.

Example

```
print(round(3.4653433, 2))  
print(round(3.7653433))  
print(round(5))
```

Output

```
3.47  
4  
5
```

max()

Used to find the largest value.

Example

```
print(max([1, 3, 5]))  
print(max((1, 3, 5)))  
print(max({1, 3, 5}))  
print(max(1, 3, 5))
```

Output

```
5  
5  
5  
5
```

sum()

Used to calculate total sum.

Example

```
print(sum([1, 2, 3]))  
print(sum((1, 2, 3)))  
print(sum({1, 2, 3}))
```

Output

```
6  
6  
6
```

Chapter 12: Strings in Python

Definition

A string is a data type used to store text in Python.

Strings are available in most programming languages like Python, Java, and C++.

Declaration of Strings

Single-Line Strings

```
str1 = 'Hello World'
str2 = "Hello World"
str3 = '''Hello World'''
```

Multi-Line String

```
str4 = '''I am Debottam.
I love coding.
I am learning Python.'''

print(str4)
```

Accessing Characters in Strings

1. Indexing

Indexing is used to access individual characters.

Python supports:

- Positive Indexing
- Negative Indexing

Positive Indexing

```
str1 = "Hello World"

print(str1[0])
print(str1[1])
print(str1[2])
print(str1[10])
```

Output

```
H  
e  
l  
d
```

Negative Indexing

```
str1 = "Hello World"  
  
print(str1[-1])  
print(str1[-2])  
print(str1[-3])  
print(str1[-11])
```

Output

```
d  
l  
r  
H
```

2. Slicing

Slicing is used to access multiple characters from a string.

Examples

```
str1 = "Hello World"  
  
print(str1[2:7])
```



```
print(str1[3:])  
print(str1[:8])  
print(str1[:])
```

Output

```
llo W  
lo World  
Hello Wo  
Hello World
```

Negative Slicing

```
print(str1[-5:-2])  
print(str1[-3:])  
print(str1[: -5])
```

Output

```
Wor  
rld  
Hello
```

Slicing with Step

```
print(str1[2:9:3])
```

Output

```
l r
```

Reverse a String

```
str1 = "Hello World"

print(str1[::-1])
```

Output

```
dlroW olleH
```

Strings are Immutable

Strings cannot be changed after creation.

Example

```
str1 = "Hello"

str1[2] = "X"
```

This gives an error because strings are immutable.

Operators in Strings

1. Concatenation

Used to join strings.

Example

```
s1 = "Hello"  
s2 = "World"  
  
print(s1 + s2)
```

Output

```
HelloWorld
```

2. Repetition

Used to repeat strings.

Example

```
s = "Hello"  
  
print(s * 3)
```

Output

```
HelloHelloHello
```

Relational Operators

Used to compare strings.

Example

```
print("apple" == "apple")  
print("apple" < "banana")
```

Output

```
True  
True
```

Membership Operator

Checks whether a substring exists.

Example

```
print("Hell" in "Hello World")  
print("h" in "Hello")
```

Output

```
True  
False
```

Loop Through String

Example

```
for i in "Hello":  
    print(i, end="-")
```

Output

```
H-e-l-l-o-
```

Built-in Functions for Strings

Function	Purpose	Example	Output
<code>sorted()</code>	Sort characters	<code>sorted("hello")</code>	<code>['e','h','l','l','o']</code>
<code>max()</code>	Largest value	<code>max("hello")</code>	<code>'o'</code>
<code>min()</code>	Smallest value	<code>min("hello")</code>	<code>'e'</code>
<code>len()</code>	Length of string	<code>len("hello")</code>	<code>5</code>

String Functions

1. Change Case Functions

Function	Example	Output
<code>capitalize()</code>	<code>"hello".capitalize()</code>	<code>"Hello"</code>
<code>lower()</code>	<code>"HELLO".lower()</code>	<code>"hello"</code>
<code>upper()</code>	<code>"hello".upper()</code>	<code>"HELLO"</code>
<code>title()</code>	<code>"hello world".title()</code>	<code>"Hello World"</code>
<code>swapcase()</code>	<code>"Hello".swapcase()</code>	<code>"hELLO"</code>

2. Checking Functions

Function	Example	Output
<code>islower()</code>	<code>"hello".islower()</code>	<code>True</code>
<code>isupper()</code>	<code>"HELLO".isupper()</code>	<code>True</code>
<code>isalpha()</code>	<code>"abc".isalpha()</code>	<code>True</code>
<code>isdigit()</code>	<code>"123".isdigit()</code>	<code>True</code>
<code>isalnum()</code>	<code>"abc123".isalnum()</code>	<code>True</code>
<code>isspace()</code>	<code>" ".isspace()</code>	<code>True</code>

3. Searching Functions

Function	Example	Output
<code>find()</code>	<code>"hello".find("l")</code>	<code>2</code>
<code>rfind()</code>	<code>"hello".rfind("l")</code>	<code>3</code>
<code>count()</code>	<code>"banana".count("a")</code>	<code>3</code>

4. Replace and Remove Spaces

Function	Example	Output
<code>replace()</code>	<code>"hello".replace("l", "x")</code>	<code>"hexxo"</code>
<code>strip()</code>	<code>" hi ".strip()</code>	<code>"hi"</code>
<code>lstrip()</code>	<code>" hi".lstrip()</code>	<code>"hi"</code>
<code>rstrip()</code>	<code>"hi ".rstrip()</code>	<code>"hi"</code>

5. Split and Join

`split()`

Converts string into list.

```
print("a b c".split())
```

Output

```
['a', 'b', 'c']
```

`join()`

Joins list elements into a string.

```
print(" ".join(['a', 'b', 'c']))
```

Output

```
a b c
```

sorted() VS sort()

sorted()	sort()
Temporary sorting	Permanent sorting
Returns new list	Changes original list

Example of sorted()

```
L = "126435"

print(sorted(L))
print(L)
```

Output

```
['1', '2', '3', '4', '5', '6']
126435
```

Example of sort()

```
L = [1, 2, 6, 4, 3, 5]

L.sort()
```

```
print(L)
```

Output

```
[1, 2, 3, 4, 5, 6]
```

Important Point

- Built-in functions use:

```
function(object)
```

Example:

```
sorted("Hello")
```

- String methods use dot notation:

```
object.method()
```

Example:

```
"Hello".upper()
```

Coding Task

Convert String to Title Case Without `title()`

```
s = "how are you"
```



```
l = s.split()

L = []

for i in l:
    j = i.capitalize()
    L.append(j)

S = " ".join(L)

print(S)
```

Output

```
How Are You
```

Chapter 13: Lists in Python

What is a List?

- A list is a built-in data type in Python.
- It stores multiple items in a single variable.
- Lists are:
 - Ordered
 - Mutable (can be changed)
 - Can store different data types

Example

```
my_list = [1, "hello", 3.14, True]
```

List vs Array

Feature	List	Array
Data Types	Can store mixed data types	Stores same data type
Flexibility	More flexible	More memory efficient

Creating Lists

Empty List

```
a = []
```

List of Numbers

```
nums = [1, 2, 3]
```

Mixed Data Types

```
mixed = [1, "hello", True]
```

Nested List

```
nested = [[1, 2], [3, 4]]
```

Accessing List Elements

Indexing

```
my_list = [10, 20, 30, 40]

print(my_list[0])
print(my_list[-1])
```

Output

```
10
40
```

Slicing

```
print(my_list[1:3])
print(my_list[:2])
```

Output

```
[20, 30]
[10, 20]
```

Accessing Nested List Elements

```
my_list_nested = [[1,2], 3, [4,5]]

print(my_list_nested[2][1])
```

Output

Editing List Elements

```
my_list = [10, 20, 30, 40]

my_list[1] = 25

print(my_list)
```

Output

```
[10, 25, 30, 40]
```

Adding Elements to a List

Assume:

```
a = [1, 2, 3, 4]
```

Method	Purpose	Example	Result
<code>append(x)</code>	Add item at end	<code>a.append(5)</code>	<code>[1,2,3,4,5]</code>
<code>insert(i,x)</code>	Insert at index	<code>a.insert(1,"hi")</code>	<code>[1,'hi',2,3,4]</code>
<code>extend(lst)</code>	Add multiple items	<code>a.extend([6,7])</code>	<code>[1,2,3,4,6,7]</code>

Deleting Elements from a List

Assume:

```
a = [1, 2, 3, 4]
```

Method	Purpose	Example	Result
<code>remove(x)</code>	Remove first occurrence	<code>a.remove(2)</code>	<code>[1,3,4]</code>
<code>pop(i)</code>	Remove item by index	<code>a.pop(1)</code>	<code>[1,3,4]</code>
<code>del</code>	Delete item	<code>del a[0]</code>	<code>[2,3,4]</code>
<code>clear()</code>	Remove all items	<code>a.clear()</code>	<code>[]</code>

Common List Operations

Operation	Example	Output
Concatenation	<code>[1,2] + [3,4]</code>	<code>[1,2,3,4]</code>
Repetition	<code>[1,2] * 3</code>	<code>[1,2,1,2,1,2]</code>
Membership	<code>3 in [1,2,3]</code>	<code>True</code>
Length	<code>len([1,2,3])</code>	<code>3</code>

Useful List Functions

Function	Purpose	Example	Output
<code>len()</code>	Count elements	<code>len([1,2,3])</code>	<code>3</code>
<code>sum()</code>	Total sum	<code>sum([1,2,3])</code>	<code>6</code>
<code>max()</code>	Largest value	<code>max([4,2,9])</code>	<code>9</code>
<code>min()</code>	Smallest value	<code>min([4,2,9])</code>	<code>2</code>
<code>list()</code>	Convert to list	<code>list("abc")</code>	<code>['a','b','c']</code>
<code>sorted()</code>	Sort list	<code>sorted([3,1,2])</code>	<code>[1,2,3]</code>
<code>reversed()</code>	Reverse list	<code>list(reversed([1,2,3]))</code>	<code>[3,2,1]</code>
<code>enumerate()</code>	Get index and value	<code>list(enumerate(['a','b']))</code>	<code>[(0,'a'),(1,'b')]</code>

Looping Through a List

Example

```
fruits = ['apple', 'banana', 'cherry']

for fruit in fruits:
    print(fruit, end=' ')
```

Output

```
apple banana cherry
```

Loop with Index Using `enumerate()`

```
fruits = ['apple', 'banana', 'cherry']

for i, fruit in enumerate(fruits):
    print(i, fruit)
```

Output

```
0 apple
1 banana
2 cherry
```

Coding Task

Remove Duplicates from a List

```
l = [1,3,2,4,1,5,2,4,1,2,5]

L = []

for i in l:
    if i not in L:
        L.append(i)

print(L)
```

Output

```
[1, 3, 2, 4, 5]
```

Chapter 14: Tuples, Sets, and Dictionary

Tuples

A tuple is a data type used to store multiple items.

Example

```
t = (1, 2, 3)
```

Features of Tuples

- Tuples are immutable.

- Elements cannot be added, removed, or changed.
 - Tuples are read-only data types.
 - Most operations are similar to lists.
-

Sets

A set is an unordered collection of unique elements.

Example

```
S = {1, 2, 3}
```

Rules of Sets

1. Sets do not allow duplicate values.
 2. Sets do not support indexing or slicing.
 3. Sets cannot contain mutable data types like lists.
 4. Sets are mutable themselves, so a set cannot contain another set.
-

Creating an Empty Set

```
S = set()
```

Important Note

```
S = {}
```

This creates an empty dictionary, not a set.

Dictionary

A dictionary stores data in key-value pairs.

Example

```
my_dict = {  
    "name": "Alice",  
    "age": 25  
}
```

Features of Dictionary

- Keys must be unique.
- Keys must be immutable.
- Values can be any data type.
- Dictionaries are mutable.
- Dictionaries do not support indexing like lists.
- Python 3.7+ keeps insertion order.

Nested Dictionary

A dictionary can contain another dictionary.

Example

```
student = {  
    "name": "Bob",  
    "marks": {  
        "math": 90,  
        "eng": 85  
    }  
}
```

```
}  
}
```

Accessing Dictionary Values

Dictionary values are accessed using keys.

Example

```
my_dict = {"name": "Alice"}  
  
print(my_dict["name"])
```

Output

```
Alice
```

Accessing Nested Dictionary

```
student = {  
    "name": "Bob",  
    "marks": {  
        "math": 90,  
        "eng": 85  
    }  
}  
  
print(student["marks"]["eng"])
```

Output

Common Dictionary Operations

Operation	Example	Output
Create Dictionary	<code>d = {"a":1, "b":2}</code>	<code>{'a':1, 'b':2}</code>
Access Value	<code>d["a"]</code>	<code>1</code>
Add/Modify Value	<code>d["c"] = 3</code>	<code>{'a':1, 'b':2, 'c':3}</code>
Delete Key	<code>del d["a"]</code>	<code>{'b':2}</code>
Check Key	<code>"a" in d</code>	<code>True</code>
Get with Default	<code>d.get("x",0)</code>	<code>0</code>
Get Keys	<code>d.keys()</code>	<code>dict_keys(['a', 'b'])</code>
Get Values	<code>d.values()</code>	<code>dict_values([1,2])</code>
Get Items	<code>d.items()</code>	<code>dict_items([('a',1), ('b',2)])</code>
Clear Dictionary	<code>d.clear()</code>	<code>{}</code>
Copy Dictionary	<code>d.copy()</code>	Copy created
Merge Dictionary	<code>d1.update(d2)</code>	Merged dictionary

Loop Through Dictionary

Example

```
d = {"a": 1, "b": 2}

for k, v in d.items():
    print(k, v)
```

Output

```
a 1  
b 2
```

Chapter 15: Functions in Python

What is a Function?

A function is a reusable block of code that performs a specific task.

Types of Functions

1. Built-in Functions

Examples:

- `print()`
- `len()`
- `type()`

2. User-defined Functions

Functions created using `def`

3. Lambda Functions

Small anonymous functions

Syntax of a Function

```
def function_name(parameters):  
    # code
```

```
return value
```

- `return` is optional.
- If no value is returned, Python returns `None`.

Types of Arguments in Functions

1. Positional Arguments

Arguments are passed in the same order as parameters.

Example

```
def power(n1, n2):  
    return n1 ** n2  
  
print(power(2, 3))
```

Output

```
8
```

2. Keyword Arguments

Arguments are passed using parameter names.

Example

```
def power(n1, n2):  
    return n1 ** n2
```

```
print(power(n2=2, n1=3))
```

Output

```
9
```

3. Default Arguments

Default values are used if arguments are not provided.

Example

```
def power(n1=1, n2=1):  
    return n1 ** n2  
  
print(power(2, 3))  
print(power(2))  
print(power(n2=3))
```

Output

```
8  
2  
1
```

4. Variable-Length Arguments (`args`)

Used when the number of arguments is unknown.

- `args` stores values as a tuple.

Example

```
def add(*nums):  
    print(sum(nums))  
  
add(1, 2, 3)
```

Output

```
6
```

5. Keyword Variable-Length Arguments (***kwargs**)

Used when the number of keyword arguments is unknown.

- ***kwargs** stores values as a dictionary.

Example

```
def show_info(**details):  
    for key, value in details.items():  
        print(key, value)  
  
show_info(name="Anu", age=22)
```

Output

```
name Anu  
age 22
```

Nested Functions

A function inside another function is called a nested function.

Example

```
def outer(x):  
  
    def inner(y):  
        return x + y  
  
    return inner  
  
print(outer(5)(3))
```

Output

8

Recursion

Recursion is when a function calls itself.

It uses the stack data structure.

Structure of Recursion

```
def recursive_function():  
    if condition:  
        return result  
    else:  
        return recursive_function()
```


Factorial Using Recursion

Example

```
def fact(n):  
    if n == 0 or n == 1:  
        return 1  
  
    return n * fact(n - 1)  
  
print(fact(5))
```

Output

```
120
```

Lambda Functions

A lambda function is a small anonymous function written in one line.

Syntax

```
lambda arguments: expression
```

Lambda Example

```
cube = lambda x: x ** 3  
  
print(cube(2))
```

Output

8

Lambda with Multiple Arguments

```
add = lambda a, b: a + b  
  
print(add(3, 4))
```

Output

7

Functional Tools in Python

Function	Purpose
<code>map()</code>	Transform items
<code>filter()</code>	Filter items
<code>reduce()</code>	Reduce items to one value

1. `map()`

Applies a function to every element.

Example

```
nums = [1, 2, 3, 4]  
  
squares = list(map(lambda x: x**2, nums))
```

```
print(squares)
```

Output

```
[1, 4, 9, 16]
```

2. `filter()`

Keeps elements that satisfy a condition.

Example

```
nums = [1, 2, 3, 4, 5, 6]

evens = list(filter(lambda x: x % 2 == 0, nums))

print(evens)
```

Output

```
[2, 4, 6]
```

3. `reduce()`

Combines all elements into one value.

Example

```
from functools import reduce

nums = [1, 2, 3, 4]
```

```
product = reduce(lambda x, y: x * y, nums)

print(product)
```

Output

24

Summary of Functional Tools

Function	Example	Output	Use
<code>map()</code>	<code>map(lambda x:x+1,[1,2,3])</code>	<code>[2,3,4]</code>	Transform items
<code>filter()</code>	<code>filter(lambda x:x>2,[1,2,3,4])</code>	<code>[3,4]</code>	Select items
<code>reduce()</code>	<code>reduce(lambda x,y:x+y,[1,2,3,4])</code>	<code>10</code>	Combine items

List Comprehension

A short way to create lists.

Syntax

```
[expression for item in iterable if condition]
```

Basic Example

```
squares = [x**2 for x in range(5)]

print(squares)
```

Output

```
[0, 1, 4, 9, 16]
```

List Comprehension with Condition

```
even_nums = [x for x in range(10) if x % 2 == 0]  
  
print(even_nums)
```

Output

```
[0, 2, 4, 6, 8]
```

Nested List Comprehension

```
pairs = [(x, y) for x in range(2) for y in range(3)]  
  
print(pairs)
```

Output

```
[(0, 0), (0, 1), (0, 2), (1, 0), (1, 1), (1, 2)]
```

Dictionary Comprehension

A short way to create dictionaries.

Syntax

```
{key:value for item in iterable if condition}
```

Example

```
even_squares = {x: x**2 for x in range(6) if x % 2 == 0}

print(even_squares)
```

Output

```
{0: 0, 2: 4, 4: 16}
```

Chapter 16: OOP (Object Oriented Programming)

What is OOP?

OOP is a programming style based on objects. Objects contain:

- Data (variables)
- Functions (methods)

Main Concepts of OOP

Concept	Meaning
Class	Blueprint for creating objects

Concept	Meaning
Object	Instance of a class
Constructor	<code>__init__()</code> method used to initialize objects
Attribute	Variable inside an object
Method	Function inside a class
self	Refers to the current object

Basic Example

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def greet(self):
        print(f"Hi, I'm {self.name}")

p1 = Person("Alice", 25)
p1.greet()
```

Output:

```
Hi, I'm Alice
```

Magic Methods (Dunder Methods)

Magic methods start and end with double underscores `__`.

Method	Purpose
<code>__init__</code>	Constructor
<code>__str__</code>	String representation

Method	Purpose
<code>__repr__</code>	Developer representation
<code>__len__</code>	Length of object
<code>__add__</code>	<code>+</code> operator
<code>__sub__</code>	<code>-</code> operator
<code>__mul__</code>	<code>*</code> operator
<code>__truediv__</code>	<code>/</code> operator
<code>__eq__</code>	Equality check
<code>__lt__</code>	Less than
<code>__gt__</code>	Greater than

Fraction Class Example

```
from math import gcd

class Fraction:
    def __init__(self, numerator, denominator):
        self.num = numerator
        self.den = denominator

    def __str__(self):
        return f"{self.num}/{self.den}"

    def __add__(self, other):
        new_num = self.num * other.den + other.num * self.den
        new_den = self.den * other.den
        return Fraction(new_num, new_den)
```

Instance Variables

Instance variables belong to each object separately.


```
class Dog:
    def __init__(self, name):
        self.name = name
```

Example:

```
d1 = Dog("Bruno")
d2 = Dog("Tommy")
```

Each object has its own `name`.

Static Variables (Class Variables)

Static variables are shared by all objects.

```
class Dog:
    species = "Canine"

    def __init__(self, name):
        self.name = name
```

Example:

```
print(Dog.species)
```

Static vs Instance Variables

Feature	Static Variable	Instance Variable
Belongs To	Class	Object
Shared	Yes	No
Defined	Inside class	Using <code>self</code>

Encapsulation

Encapsulation means hiding data and controlling access using methods.

```
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance

    def get_balance(self):
        return self.__balance
```

`__balance` is private.

Access Modifiers

Type	Syntax	Access
Public	<code>self.name</code>	Accessible everywhere
Protected	<code>self._name</code>	Internal use
Private	<code>self.__name</code>	Restricted access

Name Mangling

Python changes private variable names internally.

Example:

```
class MyClass:
    def __init__(self):
        self.__secret = "Hidden"
```

Actual name becomes:

`_MyClass__secret`

Types of Relationships

1. Aggregation (HAS-A)

```
class Engine:
    pass

class Car:
    def __init__(self):
        self.engine = Engine()
```

Car HAS-A Engine.

2. Inheritance (IS-A)

```
class Animal:
    def speak(self):
        print("Animal speaks")

class Dog(Animal):
    pass
```

Dog IS-A Animal.

Method Overriding

Child class replaces parent method.

```
class Animal:
    def speak(self):
        print("Animal")

class Dog(Animal):
    pass
```

```
def speak(self):  
    print("Dog")
```

Types of Inheritance

1. Single Inheritance

One child inherits one parent.

```
class Parent:  
    pass  
  
class Child(Parent):  
    pass
```

2. Multilevel Inheritance

```
class Grandparent:  
    pass  
  
class Parent(Grandparent):  
    pass  
  
class Child(Parent):  
    pass
```

3. Multiple Inheritance

```
class A:  
    pass  
  
class B:
```

```
pass

class C(A, B):
    pass
```

4. Hierarchical Inheritance

```
class Parent:
    pass

class Child1(Parent):
    pass

class Child2(Parent):
    pass
```

5. Hybrid Inheritance

Combination of multiple inheritance types.

MRO (Method Resolution Order)

Python checks parent classes from left to right.

```
class C(A, B):
    pass
```

Python checks **A** first.

super() Keyword

Used to call parent class methods.

```
class Animal:
    def __init__(self, name):
        self.name = name

class Dog(Animal):
    def __init__(self, name, breed):
        super().__init__(name)
        self.breed = breed
```

Polymorphism

Same method behaves differently in different classes.

Example:

```
class Animal:
    def speak(self):
        print("Animal")

class Dog(Animal):
    def speak(self):
        print("Dog")
```

Types:

- Method Overriding
- Operator Overloading
- Method Overloading (not directly supported in Python)